Sistema di Supporto alla Pianificazione Agricola

*Ottimizzazione della Schedulazione delle Colture tramite Predizione Meteorologica e Ricerca A**

Gruppo di lavoro

- Francesco Lorusso, 797838, f.lorusso100@studenti.uniba.it

Repository del progetto: https://github.com/francyloru/ICON-Project

A.A. 2025-2026

Introduzione

Il presente progetto affronta il problema della pianificazione ottimale delle colture in serre distribuite geograficamente su più città. Il sistema è progettato per supportare un'organizzazione che gestisce una serra per ciascuna città disponibile, con l'obiettivo di determinare, per ogni coltura, la città e il periodo di semina che minimizzano il costo energetico complessivo di riscaldamento/raffreddamento della serra.

Il costo giornaliero per una coltura in una data città è definito come la differenza in valore assoluto tra la temperatura media prevista per quel giorno e la temperatura ideale di crescita della coltura, ovvero la quantità di energia necessaria per portare la serra alle condizioni ottimali.

Il sistema è interamente configurabile: le colture (con le relative temperature ideali e durate di crescita) sono specificate in un apposito file JSON (*colture.json*), le città e gli anni di riferimento sono parametrizzati tramite un file di configurazione (*parametri.json*), e la pipeline è avviabile via interfaccia a riga di comando.

Configurazione dell'ambiente

Requisiti: Python 3.x. Dipendenze installabili tramite:

```
pip install pandas scikit-learn xgboost
```

La prima esecuzione richiede la sequenza di comandi:

1. python **main.py** --*new_dataset* (aggregazione e feature engineering del dataset)
2. python **main.py** --*find_models* (training, test e selezione del miglior modello per città)
3. python **main.py** --*find_scheduling* (esecuzione dell'algoritmo A*)

Il flag --evaluation_scheduling esegue il benchmark di scalabilità dell'algoritmo A*.

Sommario

Più precisamente, il sistema costituisce un Knowledge-Based System (KBS) per la pianificazione agricola ottimizzata ed è composto da due moduli principali che integrano competenze complementari di Intelligenza Artificiale. Il primo modulo affronta un problema di apprendimento supervisionato: a partire da serie storiche di dati meteorologici giornalieri, vengono addestrati modelli di regressione (Regressione Lineare, Random Forest e XGBoost) in grado di stimare la temperatura media giornaliera per ogni città nell'anno target. Il secondo modulo affronta un problema di ricerca nello spazio degli stati: note le previsioni di temperatura, un algoritmo A* (con funzione euristica ammissibile) determina la schedulazione ottimale delle colture nelle serre, minimizzando il costo energetico complessivo di climatizzazione.

I due moduli sono integrati in una pipeline sequenziale: l'output del modulo di apprendimento (le temperature predette per ogni giorno dell'anno target 2026) costituisce l'input del modulo di ricerca, che le utilizza per costruire la matrice di costi su cui opera A*. La conoscenza del dominio è definita in due livelli distinti: a livello di apprendimento, per rappresentare la periodicità stagionale della temperatura è usata una codifica ciclica (sin/cos) dei giorni dell'anno e la rappresentazione, per ogni giorno, della temperatura media registrata nello stesso giorno nell'anno precedente (tutto nel file *dati/datase_meteo_unificato.csv*); a livello di ricerca, la conoscenza agronomica delle colture (temperatura ideale, durata del ciclo vegetativo) è esternalizzata in un file di configurazione JSON (appunto *colture.json*) ed è combinata con le previsioni meteorologiche per definire la funzione di costo nello spazio a stati.

Elenco argomenti di interesse

- **Regressione e Classificazione Lineari [2]**, **Modelli Compositi: Random Forest [3]** e **Boosting [4]** tutti paragrafi dalla medesima sezione di *Apprendimento Supervisionato* (per l'apprendimento e predizione dei dati meteo)
- **Ricerca Informata (Euristica): Ricerca A* [1]**, dalla sezione di *Ricerca su Spazi a Stati* (per la ricerca dello scheduling ottimale delle colture, con formalizzazione dello spazio degli stati, funzione di costo incrementale e funzione euristica ammissibile).

Sezione 1 — Apprendimento Supervisionato per la Predizione Meteorologica

Sommario

Il sotto-problema di apprendimento consiste nel stimare la temperatura media giornaliera (TMEDIA °C) di una data città per ogni giorno dell'anno target (2026), a partire da dati storici giornalieri raccolti dal sito **ilmeteo.it** (al seguente link). I dati grezzi sono disponibili su base mensile e per città, nel formato CSV descritto nella sezione successiva.

Vengono addestrati tre distinti regressori per ciascuna delle tre città (nel nostro caso Bari, Lecce e Potenza): Regressione Lineare, Random Forest e XGBoost. Per ogni città viene selezionato e mantenuto in produzione il modello che minimizza lo score composito da $RMSE + 0.3 \cdot \sigma_residui$ calcolato sull'anno di test (2025).

Dataset e Feature Engineering

Struttura del dato grezzo

I file mensili grezzi (percorso: *dati/dati_meteo_separati_csv/*) hanno la seguente struttura per osservazione giornaliera:

Feature	Tipo	Descrizione
LOCALITA	Categorica	Nome della città (es. Bari)
DATA	Data (GG/MM/AAAA)	Data di riferimento del record
TMEDIA °C	Numerica	Temperatura media giornaliera — variabile target
TMIN °C / TMAX °C	Numerica	Temperature minima e massima giornaliere
UMIDITA %	Numerica	Umidità relativa media
VENTOMEDIA km/h	Numerica	Velocità media del vento
PRESSIONEMEDIA mb	Numerica	Pressione atmosferica media
PIOGGIA mm	Numerica	Precipitazioni cumulate
FENOMENI	Testuale	Descrizione dei fenomeni meteo

Le colonne *PUNTORUGIADA °C*, *VISIBILITA m, VENTOMAX km/h*, *RAFFICA km/h* e *PRESSIONESLM mb* vengono eliminate in quanto ridondanti o scarsamente informative per il task di predizione della temperatura media.

Pipeline di Feature Engineering

Il modulo *dati/**gestore.py*** esegue le seguenti trasformazioni per rendere i dati compatibili con i modelli di regressione:

- **Separazione della data:** il campo DATA (GG/MM/AAAA) viene scomposto in tre colonne intere ANNO, MESE, GIORNO.
- **Codifica ciclica del giorno dell'anno (SIN_GIORNO, COS_GIORNO):** il giorno dell'anno $J \in [1,365]$ viene proiettato su un cerchio unitario tramite le trasformazioni $SIN_GIORNO = \sin(2\pi \cdot J/365.25)$ e $COS_GIORNO = \cos(2\pi \cdot J/365.25)$. Questa codifica è necessaria per consentire ai modelli lineari e ad albero di apprendere la periodicità stagionale della temperatura senza discontinuità: il 31 dicembre e il 1° gennaio risultano metricamente vicini nello spazio delle feature, a differenza di quanto accadrebbe con il solo indice intero.
- **Temperatura media dell'anno precedente (TEMPERATURA_MEDIA_ANNO_PRECEDENTE):** per ogni osservazione (città, mese, giorno) viene aggiunta come feature la TMEDIA dello stesso giorno dell'anno precedente. Questo

cattura la climatologia locale specifica del periodo e fornisce al modello un forte segnale a priori, riducendo la dipendenza dalla sola componente ciclica (cioè il giorno).

- **Gestione dei valori mancanti:** le righe con valori NaN nelle feature o nella target vengono rimosse tramite *dropna()* prima dell'addestramento.

Dataset unificato e feature finali

Il dataset unificato (*dati/dataset_meteo_unificato.csv*) contiene le seguenti colonne per ogni osservazione giornaliera:

Feature	Ruolo	Descrizione
LOCALITA	Identificatore	Città di riferimento
ANNO	Feature di input	Anno intero (es. 2020)
MESE	Feature ausiliaria	Mese (1-12), non usata direttamente dai modelli
GIORNO	Feature ausiliaria	Giorno del mese, non usato direttamente
SIN_GIORNO	Feature di input	Componente sinusoidale della stagionalità
COS_GIORNO	Feature di input	Componente cosinusoidale della stagionalità
TEMPERATURA_MEDIA_ANNO_PRECEDENTE	Feature di input	TMEDIA del giorno corrispondente nell'anno precedente
TMEDIA °C	Target (output)	Temperatura media giornaliera da predire
TMIN °C, TMAX °C, UMIDITA %, VENTOMEDIA km/h, PRESSIONEMEDIA mb, PIOGGIA mm, FENOMENI	Feature ausiliarie	Presenti nel dataset ma non usate dai modelli di predizione

Il vettore di feature effettivamente utilizzato per l'addestramento e l'inferenza è pertanto X = [ANNO, SIN_GIORNO, COS_GIORNO, TEMPERATURA_MEDIA_ANNO_PRECEDENTE] $\in \mathbb{R}^4$, e la variabile target è y = TMEDIA °C $\in \mathbb{R}$.

Strumenti Utilizzati

I modelli sono implementati con la libreria **scikit-learn [5]** per Regressione Lineare e Random Forest, e con la libreria **xgboost [6]** per XGBoost. La selezione degli iperparametri è affidata a **GridSearchCV** con validazione incrociata **TimeSeriesSplit** (n_splits=5), che preserva l'ordinamento temporale delle osservazioni e impedisce data leakage dal futuro al passato. I modelli addestrati vengono serializzati tramite **joblib** nella cartella *modelli/*.

Decisioni di Progetto

Split temporale e re-training

Per tutti i modelli e tutte le città si adotta il medesimo protocollo: le osservazioni con ANNO < 2025 costituiscono il training set, quelle con ANNO = 2025 il test set. Al termine della valutazione, ciascun modello viene riaddestrato sull'intero dataset (incluso il 2025) prima di essere utilizzato per le predizioni sull'anno target 2026.

Per garantire la disponibilità della feature TEMPERATURA_MEDIA_ANNO_PRECEDENTE durante l'inferenza sull'anno 2026, il sistema salva automaticamente i dati TMEDIA dell'anno 2025 per ogni città in file CSV dedicati (*dati/dati_ultimo_anno/*).

Iperparametri e Grid Search

La scelta degli iperparametri è interamente automatizzata tramite GridSearchCV. Le griglie di ricerca per ogni modello sono le seguenti:

Modello	Iperparametro	Valori esplorati	Motivazione
Linear Regression	fit_intercept	[True, False]	Verifica se un termine di bias migliora il fit; in serie storiche climatiche l'intercetta è quasi sempre necessaria
Random Forest	n_estimators	[100, 200]	Numero di alberi: 100 è sufficiente per dataset di questa dimensione, 200 come trade-off bias-varianza
Random Forest	max_depth	[5, 10, None]	Profondità massima: None consente alberi completi, 5 e 10 regolarizzano per prevenire overfitting
Random Forest	min_samples_split	[2, 5]	Soglia minima per lo split: valori bassi aumentano la complessità
Random Forest	min_samples_leaf	[1, 2]	Minimo campioni per foglia: regolarizza la varianza delle predizioni
XGBoost	n_estimators	[100, 300]	Numero di boosting rounds; valori più alti migliorano il fit ma aumentano il rischio di overfitting
XGBoost	max_depth	[3, 6, 10]	Profondità degli alberi base; valori bassi (3) aumentano la regolarizzazione
XGBoost	learning_rate	[0.01, 0.1, 0.2]	Shrinkage per ogni step; valori bassi richiedono più alberi ma generalizzano meglio
XGBoost	subsample	[0.8, 1.0]	Frazione del training set usata per ogni albero; 0.8 introduce stochasticità e riduce l'overfitting

La metrica di ottimizzazione per la GridSearch è **neg_root_mean_squared_error**. Lo score finale per la selezione del modello migliore per città è definito come: $score = RMSE_test + 0.3 \cdot \sigma_residui$, ove $\sigma_residui$ è la deviazione standard degli errori di predizione sul test set e $\alpha=0.3$ è il peso assegnato alla componente di variabilità. Il valore $\alpha=0.3$ è scelto per dare maggiore peso all'*RMSE* rispetto alla deviazione standard: nel contesto specifico di questo sistema, un errore sistematico si cumula linearmente nei costi aggregati su 90-120 giorni calcolati da A*, mentre errori casuali tendono a compensarsi, rendendo il bias più dannoso della dispersione.

I risultati completi della grid search per ogni modello e città sono archiviati in file CSV nella cartella *dati/parametri/* e possono essere utilizzati per analisi post-hoc del comportamento dell'errore al variare degli iperparametri.

Valutazione

Tutti e tre i modelli vengono valutati sull'anno 2025 (test set comune) per ciascuna delle tre città. La tabella seguente riporta le metriche RMSE e σ_residui:

```
============================================================
   MATRICE DI CONFRONTO — RMSE (anno test: 2025)
============================================================
                    Bari  Lecce  Potenza
xgboost            2.456  2.604    3.041
random_forest      2.471  2.667    3.031
linear_regression  2.369  2.650    3.165
============================================================


============================================================
   MATRICE DI CONFRONTO — STD DEV RESIDUI (anno test: 2025)
============================================================
                    Bari  Lecce  Potenza
xgboost            2.402  2.582    2.970
random_forest      2.444  2.667    2.990
linear_regression  2.368  2.626    2.925
============================================================
```

Figura 1 — Matrici di confronto RMSE e deviazione standard dei residui (anno test: 2025)

I valori di RMSE compresi tra 2.37°C e 3.17°C sono **coerenti** con la complessità del task: la temperatura giornaliera presenta **variabilità ad alta frequenza** (eventi meteorologici locali, escursioni termiche anomale) che nessun modello puramente basato su stagionalità e climatologia storica può catturare completamente. Il bias medio risulta prossimo a zero per tutti i modelli, confermando l'assenza di distorsioni sistematiche.

In base allo score composito, la selezione automatica produce la seguente associazione città-modello:

Città	Modello selezionato	RMSE (°C)	σ_residui (°C)	Score
Bari	Linear Regression	2.369	2.368	3.079
Lecce	XGBoost	2.604	2.582	3.379
Potenza	Random Forest	3.031	2.990	3.928

I grafici seguenti confrontano le temperature reali registrate nel 2025 con quelle predette dai tre modelli per ciascuna città.

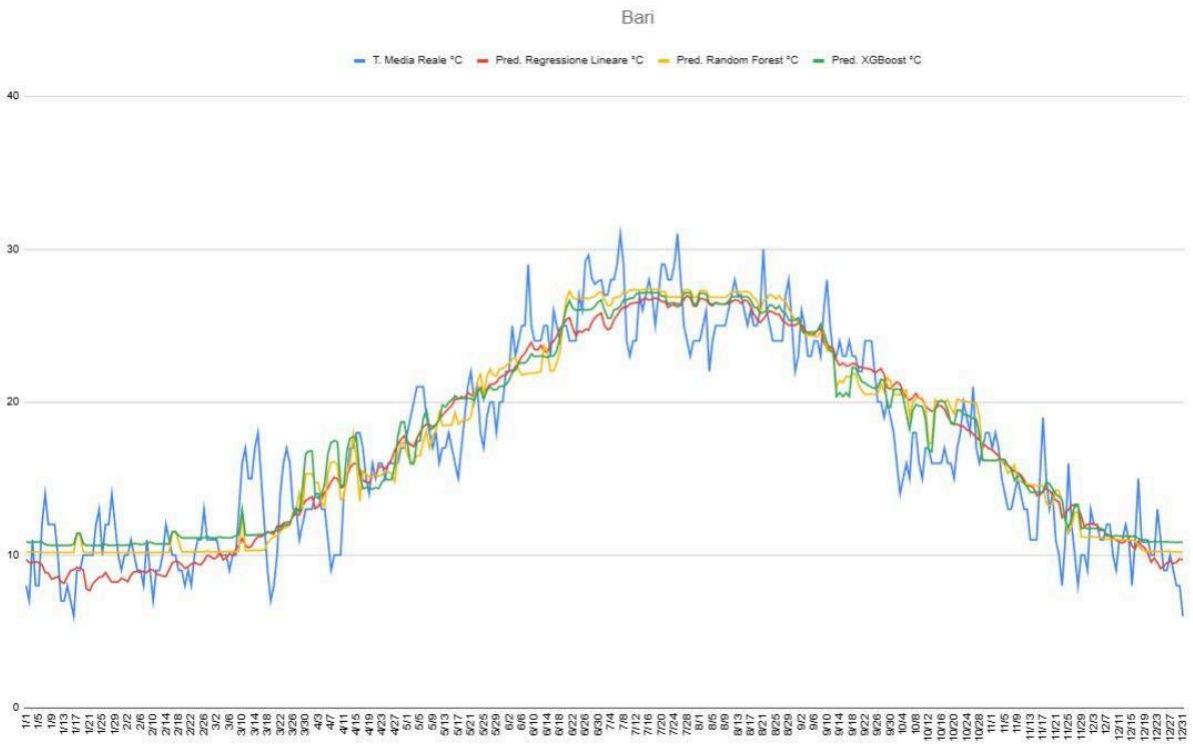

Figura 2 — Bari: confronto temperature reali vs. predette (2025)

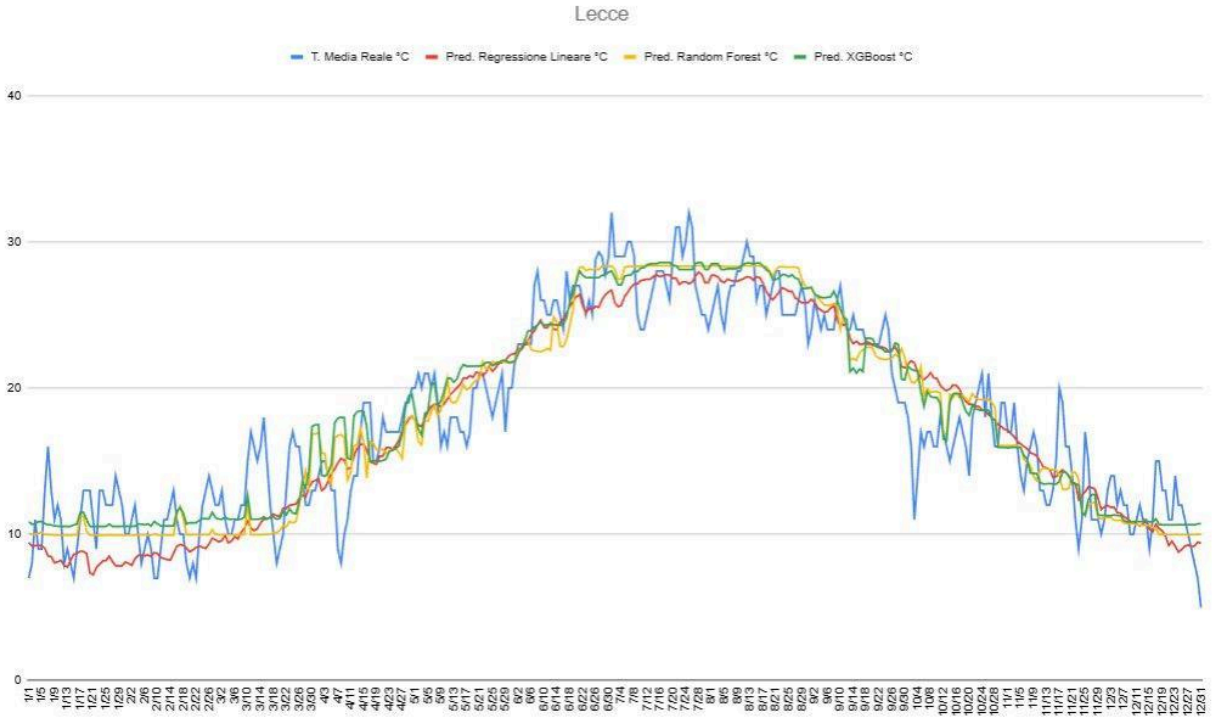

Figura 3 — Lecce: confronto temperature reali vs. predette (2025)

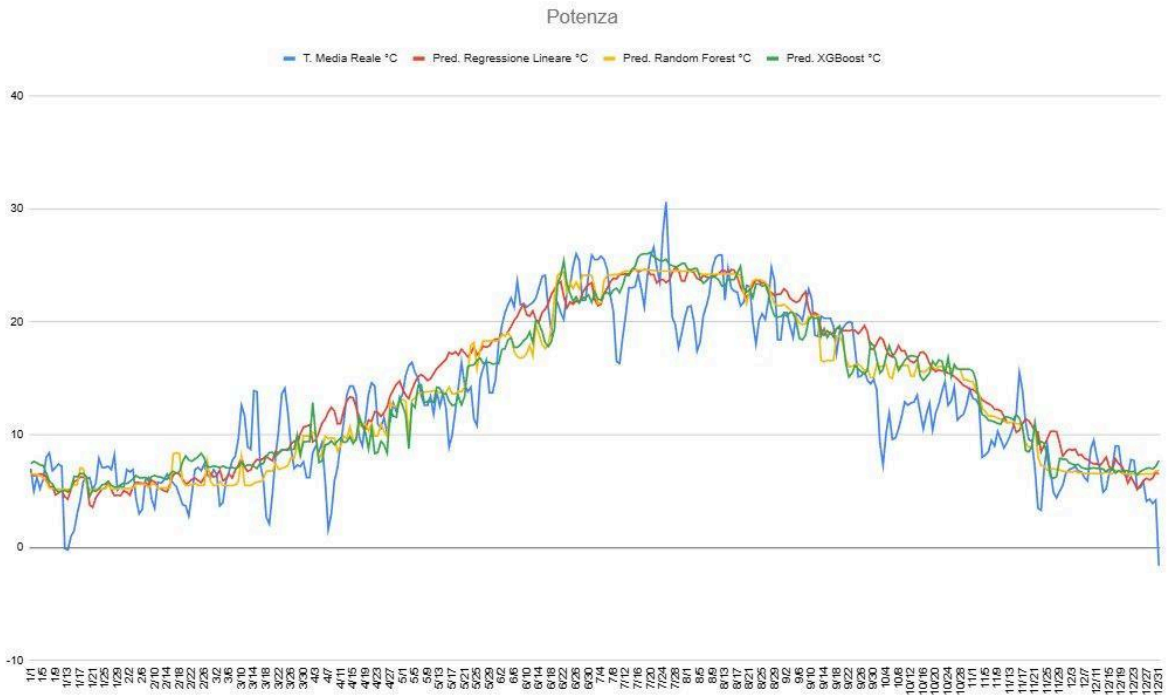

Figura 4 — Potenza: confronto temperature reali vs. predette (2025)

I grafici mostrano come tutti e tre i modelli approssimino efficacemente il trend stagionale annuale, con le predizioni che seguono fedelmente il profilo sinusoidale della temperatura. La componente non catturata

corrisponde alle oscillazioni ad alta frequenza (singoli eventi meteorologici), che per definizione non sono prevedibili da feature puramente climatologiche. La Regressione Lineare tende a produrre predizioni più regolari (minore std_dev), mentre XGBoost e Random Forest presentano maggiore variabilità ma catturano meglio alcuni picchi locali.

Sezione 2 — Ricerca con Euristica A* per lo Scheduling delle Colture

Sommario

Il problema di scheduling consiste nel determinare, per ciascuna coltura, la città e il giorno di inizio della coltivazione che minimizzano il costo energetico totale nell'anno target, rispettando il vincolo che ogni serra (una per città) può ospitare al massimo una coltura alla volta.

Il costo di coltivare la pianta *p* nella città *c* a partire dal giorno *d* è definito come:

$$\cos to\,(p, c, d) \;=\; \sum_{k\,=\,d}^{d\,+\,durata(p)\,-\,1} |T\,(c, k) - T_{ideale}\,(p)|$$

dove T(c,k) è la temperatura media prevista nella città c al giorno k e $T_{ideale}(p)$ è la temperatura ottimale di crescita della coltura p. Il costo complessivo della schedulazione è la somma dei costi individuali di tutte le coppie (pianta, assegnazione).

Rappresentazione dello Spazio degli Stati

Lo spazio degli stati è rappresentato come segue:

- **Stato:** coppia *(disp, piante_rimanenti)*, dove *disp* è un vettore di lunghezza |C| (|C| = numero di città) in cui *disp[i]* indica il primo giorno in cui la serra della città *i* è disponibile, e *piante_rimanenti* è l'insieme delle colture non ancora pianificate.
- **Stato iniziale:** *disp* = (0, 0, ..., 0), *piante_rimanenti* = (tutte le colture).
- **Stato goal:** piante_rimanenti = (), indipendentemente dal valore di disp.
- **Azione:** assegnare la prossima coltura *p* alla città *c* con data di inizio:

  $$d\star \;=\; \underset{d\geq disp[c]}{\arg\min}\; \cos to\,(p, c, d)$$

  Ogni azione aggiorna *disp[c]* ← *d** + *durata(p)* e rimuove p da piante_rimanenti.
- **Funzione di costo incrementale g:** somma cumulativa dei costi delle azioni eseguite fino allo stato corrente.

Pre-calcolo dei Costi (Lookup Table)

Per evitare il ricalcolo ripetuto durante la ricerca, prima di avviare A* viene costruita una **lookup table** COSTI_PRECALCOLATI[pianta][città][giorno_start] che contiene il costo precalcolato per ogni combinazione ammissibile (pianta, città, giorno di inizio). Le combinazioni in cui il periodo di crescita sforerebbe la fine dell'anno ricevono costo = +∞ (dal momento che abbiamo definito le predizioni solo per l'anno venturo e non oltre). La complessità di questa fase è $O(P \cdot C \cdot D^2)$ con D=365, P = numero di colture e C = numero di città.

Funzione Euristica

La funzione euristica h(s) è definita come la somma, per ogni pianta rimanente, del costo minimo globale su tutte le città e tutti i giorni possibili, ignorando i vincoli di disponibilità delle serre e i conflitti tra piante:

$$h\,(s) \;=\; \sum_{p\,\epsilon\,piante_r imanenti} \;\; \min_{c\,\epsilon\,C,\;d\,\epsilon\,[0,364]} \text{costo}\,(\mathrm{p}, \mathrm{c}, \mathrm{d})$$

L'euristica è **ammissibile** perché non sopravvaluta mai il costo reale: ignorare i vincoli di disponibilità corrente può solo ridurre il costo minimo rispetto al caso vincolato, dunque h(s) ≤ h*(s) per ogni stato s. È inoltre consistente (monotona), poiché il costo di transizione da uno stato al successivo non può ridurre il lower bound in modo tale da violare la disuguaglianza triangolare h(s) ≤ c(s, a, s') + h(s'). La consistenza garantisce che A* non riesplori mai stati già espansi.

La scelta di questa euristica è motivata dalla sua semplicità di calcolo — O(1) per pianta tramite lookup table — e dalla sua ammissibilità formale. Il suo limite principale, discusso nella sezione di valutazione, è la lassità: al crescere del numero di colture, il gap tra h(s) e h*(s) aumenta, incrementando il numero di nodi esplorati.

Struttura del nodo e gestione delle pareggi

Ogni nodo nella priority queue è rappresentato dalla tupla (f, g, contatore, disp, piante_rimanenti, storia), dove f = g + h. Il contatore è un intero crescente inserito come terzo elemento per risolvere le situazioni di parità su f e g: in questo modo heapq non tenta mai di confrontare tuple o dizionari, evitando eccezioni TypeError in Python.

Strumenti Utilizzati

L'algoritmo A* è implementato in Python, utilizzando il modulo **heapq** della libreria standard per la gestione della priority queue a complessità O(log N) per inserimento ed estrazione. Non sono state utilizzate librerie esterne specifiche per la ricerca.

Decisioni di Progetto

La struttura dello spazio degli stati esplora esplicitamente tutte le permutazioni nell'assegnazione delle colture alle città: per ogni nodo espanso, vengono generate |piante_rimanenti| × |C| azioni figlio. Questa scelta garantisce la completezza della ricerca e, insieme all'ammissibilità dell'euristica, l'ottimalità della soluzione restituita.

La ricerca assegna a ogni coltura il giorno con costo minimo disponibile (best greedy local dentro ogni ramo), non necessariamente il minimo globale: questo è corretto perché A* esplora tutti i rami e confronta i costi cumulativi tramite g, non la scelta locale.

Le colture e le loro proprietà (durata in giorni, temperatura ideale di crescita) sono esternalizzate nel file *colture.json*, rendendo il sistema completamente configurabile senza modifiche al codice. I valori di temperatura ideale di crescita e di durata media del ciclo vegetativo per ciascuna coltura sono stati ricavati da fonti bibliografiche disponibili online: **Patate [8]**, **Zucche [9] Pomodori [10]**, **Carote [11]**. Analogamente, le città e gli anni di riferimento sono parametrizzati in *parametri.json*.

Valutazione

L'algoritmo è stato valutato al crescere del numero di città (C) e colture (P) tramite il modulo valuta_a_star.py, che misura: tempo di esecuzione, nodi esplorati (*visited_states*), nodi generati (*inserzioni nella coda*), energia della soluzione, lower bound euristico iniziale e gap percentuale. La tabella seguente riporta i risultati completi:

C	P	Tempo(s)	N.Esp.	N.Gen.	Energia	L.Bound	Gap%	ms/nodo	Gen/Esp
1	1	0.0000	1	2	89.3	89.3	0.00	0.0462	2.00
1	2	0.0001	3	4	582.4	581.8	0.10	0.0246	1.33
1	3	0.0001	8	8	N/A	N/A	N/A	N/A	N/A
1	4	0.0002	10	10	N/A	N/A	N/A	N/A	N/A
2	1	0.0000	1	3	89.3	89.3	0.00	0.0455	3.00
2	2	0.0001	3	8	491.7	491.7	0.00	0.0477	2.67
2	3	0.0005	9	26	685.2	608.3	12.64	0.0532	2.89
2	4	0.0022	45	96	1244.8	1050.2	18.52	0.0487	2.13
3	1	0.0001	1	4	89.3	89.3	0.00	0.0567	4.00
3	2	0.0002	3	12	491.7	491.7	0.00	0.0748	4.00
3	3	0.0009	9	41	685.2	608.3	12.64	0.0988	4.56
3	4	0.0031	22	111	1057.5	980.6	7.84	0.1387	5.05

*Figura 5 — Risultati del benchmark di scalabilità dell'algoritmo A**

I risultati evidenziano tre aspetti rilevanti. In primo luogo, i tempi di esecuzione rimangono dell'ordine dei millisecondi in tutti gli scenari testati (massimo 3.1ms per C=3, P=4), confermando che l'algoritmo è praticabile ben oltre la dimensione del problema corrente. In secondo luogo, la crescita dei nodi esplorati è controllata: il passaggio da P=3 a P=4 con C=2 produce il salto più accentuato (da 9 a 45 nodi, fattore ×5), coerente con la crescita fattoriale attesa P! × C^P, ma i valori assoluti restano trattabili. In terzo luogo, gli scenari con una sola città e P≥3 colture producono N/A in quanto l'impossibilità di schedulare colture con durate cumulate superiori a 365 giorni non è un fallimento dell'algoritmo ma un vincolo fisico del problema.

Il gap percentuale — distanza tra la soluzione trovata e il lower bound euristico — cresce da 0% con una o due colture fino al 18.52% per C=2, P=4. Questo conferma che l'euristica è lassa nelle istanze con più colture, inducendo A* a esplorare nodi non necessari. Il gap scende a 7.84% per C=3, P=4, poiché un maggior numero di città offre più flessibilità e la soluzione trovata si avvicina al lower bound. La metrica ms/nodo, indicatore della salute interna della struttura dati, cresce moderatamente passando da circa 0.03ms (C=1) a 0.14ms (C=3, P=4), segnalando che la heap non degrada significativamente nelle istanze testate.

Il rapporto Gen/Esp (nodi generati per nodo esplorato) cresce da 1.33 a 5.05 all'aumentare della complessità, indicando che il pruning tramite closed set diventa progressivamente più efficace: A* genera molti candidati alternativi ma ne scarta la maggior parte prima di espanderli.

Una possibile estensione per migliorare la qualità dell'euristica — mantenendone l'ammissibilità — consiste nel calcolare il lower bound rispettando il vettore disp corrente per ogni città, anziché ignorarlo. Questo produrrebbe una stima più stretta del costo futuro, riducendo il gap e il numero di nodi esplorati nelle istanze multi-coltura.

Il piano ottimale trovato dall'algoritmo A* per l'anno 2026, con C=3 città (Bari, Lecce, Potenza) e P=4 colture (Zucche, Patate, Pomodori, Carote), è il seguente:

```
=== PIANO OTTIMALE TROVATO ===
  - Anno di riferimento: 2026
  - Energia Totale Stimata: 1057.5 unità termiche

                        Riepilogo:
-------------------------------------------------------------------------
|  COLTURA    |    CITTÀ    |          PERIODO OTTIMALE          | COSTO |
-------------------------------------------------------------------------
|  Zucche     |  Bari       | 14 June          -> 12 September   |  89.3 |
|  Patate     |  Lecce      | 03 February      -> 03 June        | 402.4 |
|  Pomodori   |  Lecce      | 05 June          -> 23 September   | 193.5 |
|  Carote     |  Potenza    | 26 May           -> 23 September   | 372.3 |
-------------------------------------------------------------------------
```

Figura 6 — Piano ottimale di scheduling delle colture per l'anno 2026 (energia totale: 1057.5 unità termiche)

L'energia totale di 1057.5 unità termiche rappresenta la stima del costo complessivo di climatizzazione per far crescere le quattro colture nelle condizioni termiche ottimali. Le Patate sono assegnate a Lecce nel periodo febbraio-giugno, coerentemente con la loro temperatura ideale di 17°C che Lecce approssima meglio nelle prime settimane dell'anno rispetto alle altre città. Le Zucche e i Pomodori (temperatura ideale 25°C) vengono pianificati nei mesi estivi, quando le temperature delle città meridionali si avvicinano al loro ottimo.

Conclusioni

Il sistema realizzato integra due componenti di Intelligenza Artificiale — apprendimento supervisionato e ricerca con euristica — in una pipeline coerente per la pianificazione agricola ottimizzata. I modelli di regressione producono previsioni di temperatura con RMSE compreso tra 2.37°C e 3.17°C sull'anno di test 2025, una precisione adeguata per il task downstream di scheduling, dove i costi vengono aggregati su periodi di 90-120 giorni attenuando l'impatto degli errori puntuali. L'algoritmo A* trova la schedulazione ottimale in pochi millisecondi per la configurazione corrente, con garanzie formali di ottimalità derivanti dall'ammissibilità dell'euristica.

I principali limiti del sistema sono due. Sul lato predittivo, la feature TEMPERATURA_MEDIA_ANNO_PRECEDENTE introduce una dipendenza dai dati dell'anno precedente che deve essere aggiornata annualmente; in caso di mancanza di dati storici per una nuova città, il sistema non può operare senza intervento manuale. Sul lato della ricerca, l'euristica adottata diventa progressivamente lassa all'aumentare del numero di colture (gap fino al 18.52% per P=4), aumentando il numero di nodi esplorati rispetto all'ottimo teorico.

Possibili sviluppi futuri includono: (1) l'implementazione di un'euristica più stretta che rispetti il vettore di disponibilità corrente delle serre, riducendo il gap senza compromettere l'ammissibilità; (2) l'estensione del modello di costo per includere vincoli agronomici più realistici (rotazione delle colture, vincoli di irrigazione, costi fissi di setup); (3) l'integrazione di previsioni meteorologiche ufficiali per l'anno target in sostituzione delle predizioni ML, qualora disponibili, per aumentare l'accuratezza del sistema nelle settimane imminenti.

Riferimenti Bibliografici

[1] Artificial Intelligence 3E, Foundation of Computational Agent: A* – https://artint.info/3e/...

[2] Artificial Intelligence 3E, Foundation of Computational Agent: Linear Regression – https://artint.info/3e/html/ArtInt3e.Ch7.S3....

[3] Artificial Intelligence 3E, Foundation of Computational Agent: Random Forest https://artint.info/3e/html/ArtInt3e.Ch7.S3.html#:~:text=Categorical%20Target%20Features

[4] Artificial Intelligence 3E, Foundation of Computational Agent: Gradient Boost Tree https://artint.info/3e/html/ArtInt3e.Ch7.S5.html#SS2:~:text=7.5.2%20Gradient%2DBoosted%20Trees

[5] Pedregosa et al., Scikit-learn: Machine Learning in Python, JMLR 12, pp. 2825-2830, 2011.

[6] Chen, T., & Guestrin, C., XGBoost: A Scalable Tree Boosting System, KDD 2016.

[7] Archivio Meteo, ilmeteo.it — https://www.ilmeteo.it/portale/archivio-meteo/

[8] Best Climate and Soil Conditions for Potato Farming — https://potatoinsights.com/best-climate-and-soil-conditions-for-potato-farming/#:~:text...

[9] Best Climate and Soil Conditions for Potato Farming — https://medium.com/@gardenlover/ideal-temperature-for-pumpkin-plants-f946be5bc75f

[10] Growing Tomatoes: How To Plant, Maintain, And Harvest — https://eos.com/blog/how-to-grow-tomatoes/

[11] Carrot — https://en.wikipedia.org/wiki/Carrot#:~:text=Le%20carote%20vengono,e%2070%20...